-Ingeniería de Datos-

-TPO - EduGrade Global-

*Facultad de Ingeniería y Ciencias Exactas (**FAIN**)*

Intensiva de verano

Materia: Ingeniería de Datos 2.

Profesor: Salas, Joaquin.

Alumnos: Samaan Piroli, Teodoro **|** Danko, Ivo Ignacio | Iuzzolino, Maria Florencia.

Índice

Introducción....3
Mapa de persistencia por RF (tabla)....3
Análisis técnico RF 1....4
Análisis técnico RF 2....5
Análisis técnico RF 3....6
Análisis técnico RF 4....7
Análisis técnico RF 5....8
Estructura de datos por tecnología (modelos y ejemplos)....8
Evidencia de implementación (capturas)....8
Monitoreo y operación (cómo se controla en runtime)....8

Introducción

EduGrade Global es una plataforma académica internacional para **registrar, normalizar y auditar** calificaciones. El TPO busca soportar **datos heterogéneos**, **carga masiva** y **consultas rápidas**, manteniendo **trazabilidad e inmutabilidad** (toda corrección queda como historial, sin sobrescribir).

Para eso se propone una **arquitectura políglota NoSQL** en Docker, aplicando lo visto en Ingeniería de Datos 2 (diseño *query-driven*, particionado y perfiles CAP según cada caso). En la siguiente tabla se resume la tecnología elegida para cada RF:

RF	Funcionalidad	Modelo	CAP	Partición/clave	Consistencia
RF1	Registro oficial (nota original + metadatos)	MongoDB	CP	student_id (hash) + zonas	majority / majority
RF2	Reglas conversiones versionadas (lookup)	Redis	CP	RULE:* (hash slots)	WAIT + primario
RF3	Equivalencias y trayectorias (paths)	Neo4j	CP	sin sharding (grafo completo)	ACID + quórum
RF4	Reportes históricos agregados (1M+)	Cassandra	AP	region+year(+sys)	R=ONE / W=ONE
RF5	Auditoría append-only (timeline)	Cassandra	CP	entity_id+mes	R=QUORUM / W=QUORUM

Análisis Técnico RF1 - MongoDB para Registro Académico

CAP → CP (Consistencia + Tolerancia a Particiones):
Priorizamos **Consistencia (C)** por seguridad jurídica: toda lectura debe devolver la **nota confirmada**, evitando lecturas inconsistentes. Ante fallas de red se sacrifica disponibilidad de escritura para proteger la integridad del registro oficial.

Tipo de DB → Documental (MongoDB):
Por qué: flexibilidad nativa (schema-less) para almacenar esquemas educativos distintos (ej. UK y Argentina) en una misma colección.

✓ Pros	✗ Contras
→ Flexibilidad total para distintos formatos de examen. → Lecturas rápidas con **documentos embebidos** (O(1)). → Escalabilidad mediante particionamiento.	→ Mayor consumo de disco por repetición de nombres de campos. → Complejidad en transacciones que afecten a múltiples documentos.

Trade-off:

- **Seguridad vs. Disponibilidad:** Preferimos rechazar una carga momentáneamente antes que arriesgar la veracidad del dato.
- **Velocidad vs. Duplicidad:** Usamos Desnormalización (guardar nombres de materias con la nota) para mejorar velocidad de consulta, aceptando que actualizar nombres es más costoso.
- **Particionamiento:** Utilizaremos Hashed Sharding sobre el campo student_id. Esto garantiza que las notas se distribuyan de forma uniforme entre los servidores, evitando "puntos calientes" o hotspots.

N-R-W:

- **N=** 3 → Tres réplicas (un Primario y dos Secundarios).
- **W=** 2 → Asegura que la mayoría de los nodos confirmen la escritura.
- **R=** 2 → Garantiza consistencia fuerte al leer.

Estrategia de Escalamiento a Futuro:

- **Escalamiento Horizontal:** Gracias al Hashed Sharding, si el volumen sube de 1 a 10 millones de registros, simplemente agregamos nuevos shards (servidores) al clúster.
- **Balanceo Automático:** MongoDB redistribuye **chunks** entre shards sin interrumpir el servicio, manteniendo rendimiento en lecturas y escrituras.

—grafico–

Análisis Técnico: RF2 - Redis para Conversión y Normalización

CAP → CP (Consistencia + Tolerancia a Particiones):
Justificación: Redis puede comportarse como AP en cluster, pero en RF2 se **prioriza consistencia** sobre la versión aplicada: se controla la **versión activa** y su **vigencia temporal**. Ante una partición, se prefiere **fallar la conversión** antes que aplicar una regla incorrecta.

En conversiones para reportes oficiales la consistencia es crítica. Si cambia una regla (ej. "UK A*" → "US 4.0"), el sistema debe asegurar que el reporte inmediato use la regla nueva para evitar errores legales.

Tipo de DB → Clave-Valor (Redis):
Por qué: funciona como un "diccionario" de lookup de alta frecuencia; al operar en memoria permite búsquedas O(1), clave para conversiones masivas sin cuellos de botella.

✓ Pros	✗ Contras
→ **Latencia ultra baja:** Tiempo de respuesta en microsegundos (O(1)), ideal para consultas masivas repetitivas. → **Modelo simple:** Key-Value encaja con reglas tipo diccionario.	→ **Volatilidad:** requiere persistencia (AOF/RDB) para evitar pérdida ante reinicios. → **Costo:** La RAM es más costosa que el almacenamiento en disco para volúmenes grandes.

Trade-off

- **Exactitud vs. Latencia:** se acepta el costo de lectura consistente para no normalizar con reglas obsoletas, priorizando validez legal sobre velocidad bruta.
- **Especialización vs. Complejidad:** usar Redis evita saturar la base principal (MongoDB) con tráfico de solo lectura, a cambio de administrar una tecnología adicional.

Especificaciones Técnicas

- **Sharding:** Redis Cluster con Hash Slots. Claves (ej. RULE#UK#US) se distribuyen automáticamente por hash para balancear memoria entre nodos.
- **Clustering y replicación:** replicación **Master–Slave** para alta disponibilidad: si cae el nodo principal, una réplica toma el rol manteniendo integridad de reglas.

N-R-W (simulado en Redis)

- **N = 3** → 1 maestro + 2 réplicas.
- **W** → uso de WAIT para confirmar actualización cuando se replica a la mayoría (propagación antes de confirmar).
- **R** → lecturas siempre desde el **maestro** para consistencia fuerte (CP).

Estrategia de Escalamiento

- **Escalamiento horizontal:** si crece el número de reglas internacionales, se agregan nodos al clúster.
- **Rebalanceo:** el rebalanceo de hash slots reparte la carga de memoria entre los nuevos nodos de forma transparente.

—grafico–

Análisis Técnico RF3 - Gestión de Relaciones Académicas Complejas

CAP → CP (Consistencia + Tolerancia a Particiones):
Justificación: Las equivalencias académicas son reglas“fuertes” (impactan inscripciones y correlativas). Si una equivalencia cambia (o deja de existir), el cambio debe verse **de forma inmediata** para evitar inscripciones basadas en relaciones inválidas. Ante fallas de red, se prefiere **denegar escrituras** antes que dejar el grafo incoherente (*split-brain*).

Tipo de DB: Orientada a grafos (Neo4j)

Por qué: es la opción más adecuada para analizar **trayectorias** y **caminos** entre materias. Con **Cypher** se resuelven consultas de profundidad variable(paths) (ej. “qué camino debe seguir el alumno para graduarse”) en una sola consulta, mientras que en bases documentales requeriría múltiples consultas o agregaciones más lentas.

✓ Pros	✗ Contras
Relaciones de primer nivel: navegar relaciones tiene costo constante por Index-Free Adjacency. **Integridad Referencial:** al borrar una materia, las relaciones asociadas se gestionan automáticamente, evitando “IDs huérfanos”.	**Escalabilidad Vertical:** Los grafos son difíciles de particionar, por lo que suele requerir más RAM para mantener el grafo en memoria. **Curva de Aprendizaje:** Se necesita dominar **Cypher**, distinto a SQL/JSON tradicional.

Trade-off

- **Complejidad vs performance de consulta:** se incorpora Neo4j solo para relaciones; aumenta la complejidad operativa, pero se baja el tiempo de consulta de trayectorias (de segundos a milisegundos).
- **Persistencia políglota (duplicación mínima):** en Neo4j se guarda solo el **ID** y **nombre** de la materia; el detalle pesado (historial/notas) queda en MongoDB. Esto optimiza RAM en Neo4j, pero requiere sincronización entre bases.

Especificaciones Técnicas

- **Particionamiento:** por la complejidad del particionamiento de grafos, **no se hace sharding**. Se mantiene el **grafo completo** en cada nodo para que las consultas recursivas no dependan de saltos entre servidores.
- **Clustering y replicación:** Neo4j Causal Cluster
    - **Core Servers (3):** manejan escrituras y garantizan consistencia mediante consenso.
    - **Read Réplicas:** escalan las consultas de lectura de trayectorias.

Modelo N-R-W: no aplica N-R-W ajustable. Se usan **transacciones ACID**: los cambios se confirman en el líder y se replican de forma consistente; si no se asegura escritura en la mayoría, la operación **falla** para proteger la integridad del grafo.

Estrategia de Escalamiento a futuro

- **Escalamiento Vertical:** aumentar RAM en Core Servers para soportar millones de relaciones.
- **Escalamiento Horizontal de Lectura:** agregar **Read Replicas** si crece el volumen de consultas de trayectorias.

—grafico–

Análisis Técnico: RF4 - Reportes Oficiales y Análisis Histórico.

CAP: AP (Disponibilidad + Tolerancia a Particiones).
Justificación: en análisis histórico la **disponibilidad** es prioritaria para que tableros y reportes no se queden “cargando”. Se acepta **consistencia eventual** (se sacrifica C) porque un retraso de segundos en datos estadísticos es aceptable frente a velocidad y estabilidad.

Tipo de DB: Columnar / Wide-Column (Apache Cassandra).
Por qué: es la solución para el “Big Data” del TPO: soporta **volúmenes masivos**, **escritura continua** y consultas analíticas agregadas (promedios, evoluciones) sin degradar el rendimiento.

✓ Pros	✗ Contras
Escritura masiva y lineal: por su arquitectura **LSM Trees**, inserta grandes volúmenes con alta eficiencia. **Modelado query-driven:** tablas diseñadas para el reporte final, habilitando lecturas secuenciales rápidas. **Arquitectura masterless (peer-to-peer):** sin cuellos de botella; escala al agregar nodos/servidores.	**Rigidez de consulta:** eficiente solo consultando por columnas de la **clave primaria**; no es flexible para búsquedas ad-hoc. **Duplicación de datos:** para múltiples vistas de reportes se duplican tablas, aumentando almacenamiento.

Trade-off

- **Storage vs compute:** se usa más disco (barato) para dejar el dato “pre-cocinado”, reduciendo CPU y cálculos al vuelo en dashboards.
- **Consistencia tunable:** se ajusta para un sistema “always on”, aceptando que la propagación entre réplicas no sea instantánea.

Especificaciones Técnicas

- **Sharding: Partition Key = Región + Año Académico** para agrupar físicamente datos relacionados para resolver consultas masivas con acceso secuencial.
- **Clustering y replicación**
    - **Replication Factor (N) = 3:** cada reporte queda en tres nodos para tolerar fallos sin perder disponibilidad.
    - **Masterless:** todos los nodos son equivalentes; si uno cae, otro responde las consultas.

Modelo N-R-W

- **N** = 3, cada registro del reporte se almacena en **3 nodos** distintos.
- **W** = 1, una escritura se considera exitosa cuando **al menos 1 de los 3 nodos** confirma. Esto maximiza **disponibilidad** y **velocidad de ingesta**, porque si uno o dos nodos están lentos o caídos, el sistema igual puede seguir recibiendo datos.
- **R** = 1, una lectura se responde con **la primera réplica disponible**. Esto mejora el tiempo de respuesta en tableros y reportes, evitando esperas por coordinaciones entre réplicas.

Con esta configuración se prioriza el perfil **AP**: el sistema permanece **always on** aun con fallas parciales o particiones de red. El costo aceptado es que, durante un corto período, una lectura puede devolver datos **no totalmente actualizados** si alguna réplica aún no recibió la última escritura (**consistencia eventual**), lo cual es aceptable para métricas y análisis histórico.

Estrategia de Escalamiento a Futuro:

- **Escalamiento Lineal:** si el historial crece (ej. de 1 a 100 millones), se agregan nodos al clúster. El sistema escala horizontalmente de forma predecible sin necesidad de reestructurar la base de datos.

—grafico–

Análisis Técnico: RF5 - Auditoría, Trazabilidad y Control Normativo

CAP: CP (Consistencia + Tolerancia a Particiones).
Justificación: en auditoría ministerial la “verdad” del historial es innegociable. Se prioriza **Consistencia (C)** para garantizar **no-repudio**: un auditor debe ver siempre el log completo y real. Si no se alcanza quórum, se **rechaza la escritura** para evitar rastros incompletos.

Tipo de DB: Columnar / Wide-Column (Apache Cassandra).
Por qué: es ideal para **logs inmutables** (*append-only*). Permite ingesta a alta velocidad sin degradarse y organiza los eventos en forma cronológica.

✓ Pros	✗ Contras
Escritura masiva: por **LSM Trees**, soporta escrituras secuenciales bajo carga extrema. **Series temporales:** usando timestamp como Clustering Key, los logs quedan ordenados físicamente, facilitando lecturas de historial. **TTL (Time To Live):** permite cumplir normativas (ej. retención 5 años) eliminando datos viejos automáticamente.	**Latencia de escritura:** con consistencia **QUORUM** hay unos milisegundos extra mientras se confirma mayoría. **Rigidez de esquema:** hay que definir qué campos se auditan (usuario, acción, IP, etc.), porque fuera de la clave primaria las consultas son ineficientes.

Trade-off

- **Latencia vs garantía legal:** se acepta un pequeño costo en tiempo de respuesta para garantizar que si el sistema confirma un "guardado", el rastro queda consistente en todo el clúster.

Especificaciones Técnicas

- **Sharding: Partition Key =** entity_id + mes_año.
  Evita particiones demasiado grandes y agrupa eventos por entidad(alumno/nota) para trazabilidad rápida.
- **Clustering y replicación**
  - **Replicacion (N) = 3**
  - **Clustering key:** timestamp, para asegurar orden temporal físico en disco.

Consistencia (N-R-W)

- **W = QUORUM (2)**: se confirma solo si escribe la mayoría.
- **R = QUORUM (2)**: el auditor lee la versión más reciente confirmada.
- **Regla: R + W > N** (2 + 2 > 3) → Cassandra se comporta como **CP**.

Estrategia de Escalamiento a Futuro

- **Escalamiento horizontal:** agregar nodos al anillo cuando aumente el volumen de datos; la Partition Key **entity_id + mes_año** reparte la carga y evita particiones excesivas.

- **Consulta eficiente por tiempo:** el bucket mensual y el **timestamp** como clustering key permiten reconstruir historiales por entidad con rangos temporales sin degradar performance.

- **Separación por dominio si crece el tráfico:** dividir auditoría en tablas (calificaciones / reglas / reprocesos) para mantener diseño *query-driven* y evitar mezcla de patrones de acceso.
- **Retención automática:** aplicar **TTL** de 5 años para controlar tamaño del histórico sin tareas batch pesadas, para una mejor performance y simplicidad en el mantenimiento.

—grafico–

Estrategia de Infraestructura: El Corazón de EduGrade en Sudáfrica

El despliegue en Sudáfrica se plantea como una arquitectura distribuida para asegurar soberanía del dato y resiliencia. Se definen cuatro puntos —**Ciudad del Cabo, Puerto Elizabeth, Bloemfontein y Pretoria**—, donde cada nodo cubre aproximadamente el **25% del territorio**, pero está integrado en un ecosistema global de alta consistencia.

La distribución se apoya en tres pilares:

- **Minimización de latencia geográfica:** al procesar datos cerca de su origen (vía **Zoned Sharding** en MongoDB o **Partition Keys** en Cassandra), un estudiante en Pretoria o un auditor en Ciudad del Cabo obtiene respuestas en milisegundos (O(1)).

- **Resiliencia ante desastres:** con **réplicas cruzadas** y **causal clustering**, si una sede queda fuera de servicio la “verdad” del dato queda protegida en las otras tres, manteniendo el sistema **always on** para reportes ministeriales.

- **Balance CAP adaptativo:** la red puede operar como **AP** para reportes estadísticos (RF4) y “endurecerse” a **CP** para carga de notas (RF1) y auditoría legal (RF5), evitando inconsistencias con impacto jurídico.

Con esta distribución, la persistencia políglota no solo responde a una elección de software, sino también a una ventaja física de despliegue: el sistema soporta ráfagas de escritura masiva en Pretoria mientras sigue entregando certificados oficiales con integridad ACID en el resto del país.

Monitoreo y Evaluación de Conectividad

EduGrade Global utiliza una arquitectura políglota (MongoDB, Redis, Neo4j y Cassandra). Para asegurar estabilidad operativa y facilitar el diagnóstico de fallas, se define un esquema de monitoreo que permite:

1. Verificar la conectividad real entre la aplicación y cada motor de base de datos, antes y durante las pruebas.

2. Detectar cuellos de botella mediante métricas mínimas (latencia, errores y uso de recursos).

3. Comprobar que cada requerimiento mantiene el comportamiento esperado según su perfil (por ejemplo: RF1/RF5 priorizan consistencia; RF4 prioriza disponibilidad).

Estrategia de monitoreo (qué medimos y por qué)

La estrategia se divide en 4 grupos de métricas:

1. Métricas de infraestructura

Se monitorean recursos del servidor/contenedor de cada base: **CPU, memoria, disco y red**. Se definen alertas por umbrales para anticipar saturación.

Umbrales de referencia (para documentar reglas de alerta):

- CPU usage > 80% por 5 minutos
- Memory usage > 90%
- Disk usage > 85%
- Network latency > 50ms

[ACÁ VA LA CAPTURA]: *docker stats* (o panel equivalente) mostrando CPU/RAM de MongoDB/Redis/Neo4j/Cassandra durante ejecución.

2. Métricas de aplicación

Se monitorea cómo responde el sistema “de cara a los endpoints/consultas”:

- Latencia de requests
- Throughput (cantidad de operaciones)
- Error rate
- Uso del pool de conexiones por base

Umbrales de referencia:

- Response time p95 > 100ms

- Error rate > 1%
- Connection pool > 80% utilization
- Queue depth > 100 requests

[ACÁ VA LA CAPTURA]: medición simple de tiempos (por ejemplo, ejecución repetida de una query/endpoint) + logs de errores si los hubiera.

3. Métricas de seguridad (recomendado)

Se registran y monitorean eventos como:

- intentos fallidos de autenticación
- conexiones rechazadas
- patrones anómalos de acceso

Esto es especialmente relevante para RF5 (auditoría y control normativo).

[ACÁ VA LA CAPTURA: ejemplo de log con intento fallido / acceso denegado]

4. Alertas (reglas simples)

Se definen alertas mínimas para notificar degradación del sistema:

- Error rate > 1%
- p95 > 100 ms sostenido
- Memoria > 90%
- CPU > 80% sostenido

[ACÁ VA LA CAPTURA: ejemplo de alerta o log donde se vea el evento (real o simulado)]

Evaluación de conectividad (antes de correr pruebas)

Antes de probar RFs, hacemos un “smoke test” de conectividad para confirmar servicio arriba, puertos accesibles y credenciales OK. La clase plantea factores clave: protocolo, autenticación, failover, latencia y firewalls/puertos.

1 Pruebas mínimas por motor (CLI)

- **Redis**: `redis-cli ping` → respuesta esperada `PONG`.
  [ACÁ VA LA CAPTURA]: terminal con `PONG`.
- **Neo4j**: `cypher-shell -u neo4j -p password "RETURN 1;"` → respuesta esperada `1`.
  [ACÁ VA LA CAPTURA]: resultado `1`.

- **Cassandra**: `cqlsh localhost` + si falla revisar `nodetool status`.
  [ACÁ VA LA CAPTURA]: `cqlsh` conectado.
  [ACÁ VA LA CAPTURA]: salida de `nodetool status`.
- **MongoDB**: prueba de ping mongosh `--eval "db.runCommand({ ping: 1 })"`.
  [ACÁ VA LA CAPTURA]: `{ "ok" : 1 }`.

2 Diagnóstico si algo falla

Errores comunes y qué revisar:

- Connection refused → servicio no iniciado o firewall/puerto
- Authentication failed → credenciales incorrectas
- Timeout exceeded → latencia alta / timeouts
- Invalid host → IP/DNS/host incorrecto

[ACÁ VA LA CAPTURA]: log/terminal con el error y el ajuste aplicado.

Monitoreo aplicado a cada RF (qué controlamos y qué evidencia dejamos)

RF1 — MongoDB (Registro académico, perfil CP)

Qué se controla

- Salud del servidor (conexiones, memoria, estado general)
- Índices y volumen de datos (para sostener consultas operativas)
- Estabilidad en escrituras/lecturas consistentes (perfil CP)

Evidencias

- **[ACÁ VA LA CAPTURA: panel/estado del servidor (conexiones + memoria)]**
- **[ACÁ VA LA CAPTURA: índices del collection principal + stats básicos]**

RF2 — Redis (Conversión y normalización)

Qué se controla

- Memoria (used_memory / maxmemory / fragmentation)
- Eficiencia de lookup (hits/misses)
- Latencia de operación y estabilidad del servicio

Evidencias

- **[ACÁ VA LA CAPTURA: redis-cli ping + INFO memory]**

- **[ACÁ VA LA CAPTURA: INFO stats (hits/misses)]**
- **[ACÁ VA LA CAPTURA: logs del update de regla + confirmación (si se usa WAIT)]**

RF3 — Neo4j (Equivalencias y trayectorias)

Qué se controla

- Conectividad estable al motor (Bolt / cypher-shell)
- Latencia de consultas típicas de trayectorias (paths)
- Consistencia en escrituras del grafo (si el cluster no logra consenso, la operación debe fallar)

Evidencias

- **[ACÁ VA LA CAPTURA: cypher-shell RETURN 1]**
- **[ACÁ VA LA CAPTURA: consulta de trayectoria en Neo4j Browser con tiempo de ejecución]**

RF4 — Cassandra (Reportes históricos)

Qué se controla

- Estado del cluster (nodos UP/DOWN)
- Latencia de lectura de reportes (base para tableros)
- Disponibilidad: el sistema debe responder aun con fallas parciales (perfil AP)

Evidencias

- **[ACÁ VA LA CAPTURA: nodetool status]**
- **[ACÁ VA LA CAPTURA: query de reporte (región+año) + tiempo aproximado]**
- **[ACÁ VA LA CAPTURA: prueba con un nodo caído y lectura funcionando]**

RF5 — Cassandra (Auditoría / trazabilidad)

Qué se controla

- Quórum y consistencia real (operaciones fuertes deben depender de mayoría)
- Latencia de escritura/lectura en QUORUM
- Crecimiento del log y estabilidad del particionamiento (entity_id + mes_año)

Evidencias

- **[ACÁ VA LA CAPTURA: nodetool status + ejemplo de operación QUORUM]**

- **[ACÁ VA LA CAPTURA: query de timeline ordenado por timestamp]**

Logging mínimo a nivel aplicación

Además del estado de las bases, se registran logs para auditoría y diagnóstico de performance. Cada operación relevante incluye:

- base involucrada
- operación (read/write)
- duración
- resultado (ok/error)
- identificador de entidad (cuando aplique)

[ACÁ VA LA CAPTURA: salida de logs durante una operación real (ej. conversión masiva o carga de notas)]

Checklist final de evidencias a adjuntar

1. **Consumo de contenedores** (CPU/RAM)
   [ACÁ VA LA CAPTURA: docker stats / panel]

2. **Conectividad por motor** (ping / return 1 / cqlsh / nodetool)
   [ACÁ VA LA CAPTURA: comandos y resultados]

3. **Métricas de aplicación** (p95, error rate)
   [ACÁ VA LA CAPTURA: medición de tiempos + logs]

4. **Al menos una prueba de tolerancia** (ej. nodo down en Cassandra RF4)
   [ACÁ VA LA CAPTURA: evidencia de que el reporte sigue respondiendo]

Lo mismo q arriba pero resumido en un cuadro(para ayudarnos a nosotros)

Se implementa un esquema de monitoreo para detectar cuellos de botella y asegurar estabilidad en la arquitectura políglota. Se miden métricas de infraestructura y de aplicación, con umbrales y alertas básicas.

RF	Base / Rol	Qué monitoreamos (mínimo)	Cómo lo medimos (herramientas / comandos)	Evidencia para el informe

RF 1	**MongoDB** (registro académico)	**Infra:** CPU, memoria, disco, red. **App:** latencia p95, throughput, error rate, pool de conexiones. **DB:** estado del server + análisis de índices/colecciones (conteo, tamaño, etc.).	- Ping básico Mongo: `mongo --eval "db.runCommand({ping : 1})"` (ok=1). - Diagnóstico/estado: `serverStatus` + stats/índices (ej. desde Compass o script).	**ACÁ VA LA CAPTURA:** ping OK de Mongo (ok:1). **ACÁ VA LA CAPTURA:** panel/resultado de serverStatus (memoria, conexiones) y resumen de índices.
RF 2	**Redis** (reglas de conversión)	**Infra:** CPU/RAM del contenedor. **Redis:** memoria (used/max/fragmentation) + hits/misses + comandos procesados. **Latencia:** latencia de operación (<1ms esperado).	- Salud: `redis-cli ping` → PONG. - Métricas: `INFO memory`, `INFO stats` (hits/misses). - Latencia: `redis-cli --latency`.	**ACÁ VA LA CAPTURA:** `redis-cli ping` con PONG. **ACÁ VA LA CAPTURA:** `INFO memory` y `INFO stats` (keyspace_hits / keyspace_misses). **ACÁ VA LA CAPTURA:** medición de latencia (`redis-cli --latency`).
RF 3	**Neo4j** (equivalencias/trayectorias)	**Infra:** CPU/RAM (grafos suelen ser intensivos en memoria). **App:** latencia de consultas/patrones (paths). **DB:** tiempo de traversal (orden de 10–50ms en grafos medianos, como referencia).	- Salud: `cypher-shell -u neo4j -p password "RETURN 1;"` → 1. - Medición de traversal: Neo4j Browser (y opcional APOC).	**ACÁ VA LA CAPTURA:** `RETURN 1` respondiendo 1. **ACÁ VA LA CAPTURA:** ejecución de una query de path mostrando tiempo (en Neo4j Browser).

RF 4	**Cassandra** (reportes históricos, AP)	**Infra:** CPU/disco/red (ingesta y scans por partición). **App:** latencia p95 + error rate + pool. **Cluster:** nodos activos + latencia de escritura (1–5ms cluster local como referencia).	- Salud: `cqlsh localhost` (conecta ok). - Estado cluster: `nodetool status`. - Diagnóstico de latencia: `nodetool` / (mencionar OpsCenter si quieren).	**ACÁ VA LA CAPTURA:** `cqlsh` conectado. **ACÁ VA LA CAPTURA:** `nodetool status` mostrando nodos UP. **ACÁ VA LA CAPTURA:** query de reporte por partición (región+año) + tiempo aproximado.
RF 5	**Cassandra** (auditoría/timeline, CP por configuración)	**Infra:** CPU/disco/red. **App:** p95 + error rate + pool (especialmente en escrituras de auditoría). **Cluster:** salud del ring + latencia de escritura (referencia 1–5ms local; con consistencia fuerte puede subir).	- Estado cluster: `nodetool status`. - Medición de latencia: `nodetool` (y logs de app para tiempos por operación). - Logging con contexto (duración, userId, etc.) para auditoría/diagnóstico.	**ACÁ VA LA CAPTURA:** `nodetool status` OK. **ACÁ VA LA CAPTURA:** ejemplo de lectura de timeline (por entity_id) mostrando orden temporal. **ACÁ VA LA CAPTURA:** logs de app con `duration` + `database` + `userId`.

Latencia

Tza de errores baja